ROASTMASTER AI | FULL-STACK RAG ENGINE | POSTGRESQL PGVECTOR | GEMINI 2.5 FLASH | NEXT.JS 16 + REACT 19

Technical Deep-Dive & System Architecture Interview Guide

A comprehensive technical manual and question-and-answer preparation guide for software engineering, GenAI, and RAG interview rounds based on the real production architecture of RoastMaster AI.

PRIMARY LLM	EMBEDDING MODEL	VECTOR STORAGE	EVALUATED RECALL@3
Google Gemini 2.5 Flash	**gemini-embedding-2 (768-d)**	**Aiven pgvector (HNSW)**	**96.7% (MRR: 0.936)**

1. Project Overview & Core Value Proposition

System Overview

What is RoastMaster AI?

RoastMaster AI (`roastmaster-ai`) is a dual-interface AI application (accessible via Web Stage at roastmaster-phi.vercel.app and globally on `npm` via `npx roastmaster-ai`). It acts as an unfiltered, sharp-witted standup comedian and senior staff code reviewer with zero mercy.

The Core Dual-Persona Philosophy:

- **The Savage Front-Stage Roast (1–2 Punchlines):** Provides razor-sharp, crowd-work comedy that critiques the user's code sins, architectural flaws, state loops, or bad dependencies with calibrated severity (1–10). Clichés and predictable formulas are explicitly banned via system prompt heuristics.
- **Backstage Real Talk (Constructive Advice):** Every roast is paired with genuine, high-caliber Senior Staff architectural advice to actually solve the problem.
- **Grounding via RAG (pgvector):** Users can upload PDF specifications, RFCs, or documentation. The system retrieves exact passages, cites verified page numbers, and uses their own documentation against their bad code — or cleanly refuses when information is missing.

2. Architecture & Data Flow Pipelines

Data Pipelines

Pipeline A: RAG Document Ingestion & Indexing Flow

```
[User Uploads PDF] (Max 2.5MB / 40 Pages) | ▼ [unpdf Parser] (extractText with mergePages: false -> Preserves Page Arrays) | ▼ [Sentence-Level Chunker] (Intl.Segmenter sentence boundaries -> 700-char budget, 1-sent overlap) | ▼ [Batch Embeddings Generator] (Gemini batchEmbedContents -> 50
```

```
chunks/batch, 768-d vectors) │ ▼ [Transactional PostgreSQL Storage] (BEGIN -> DELETE old chunks ->
INSERT INTO document_chunks -> COMMIT) │ ▼ [Aiven pgvector HNSW Index Ready for Cosine Retrieval
(<=>)]
```

Pipeline B: Query, 2-Stage Retrieval & Grounding Flow

```
[User Prompt: e.g., "What is laptop warranty limit?"] │ ▼ [Conversational Banter Detector] (Filters
out "huh", "lol", "roast me" from triggering doc search) │ ├─► [Broad Recall: Stage 1] ──► Gemini
Query Embedding (768-d, LRU-cached) │ │ │ ▼ │ Query Aiven pgvector (Top 15 candidates via cosine
<=>) │ │ │ (If 0 candidates returned) ▼ │ └──────────────────────────► [Hybrid Resilience Fallback]
(Fetch direct SQL chunks) │ │ ▼ ▼ [Multi-Signal Composite Reranker: Stage 2] • 40% Vector Cosine
Similarity • 25% Salient Term Density (stopword stripped) • 25% Query Keyword Coverage Ratio • 10%
Exact Phrase Match Bonus │ ▼ [Redundancy Suppression] (Jaccard token overlap > 0.65 filtered out) │
▼ [Top 3–5 High-Precision Chunks Selected] │ ▼ [LLM Context Injection] (Gemini 2.5 Flash with
System Prompt & Grounding Protocol) │ ▼ [Model Generates Structured JSON] ({ title, roast,
severity, suggestion, citations }) │ ▼ [Strict Citation Validation Engine] (Verifies citations
against real retrieved chunks, strips fakes) │ ├─► Output Streamed to Client (useCharacterReveal
Teleprompter) │ └─► [Background Fire-and-Forget] (Async embedding upsert into session_embeddings)
```

3. Core Components & Implementation Details

Code Mechanics

Component 1: Database Architecture (Aiven PostgreSQL + pgvector)

Why pgvector over standalone vector DBs? RoastMaster AI stores transactional user data, authentication IDs, sessions, multi-turn messages, and vector embeddings in the exact same relational database. This enables single-query ACID transactions, cross-table relational joins, and eliminates the synchronization lag common to dual-database setups (e.g., Postgres + Pinecone).

src/lib/db.js — Cosine Similarity Vector Search Query — PostgreSQL pgvector

```
SELECT id, file_name AS "fileName", chunk_index AS "chunkIndex", page_number AS "pageNumber", content, embedding_model AS "embeddingModel", embedding_dimension AS "embeddingDimension", 1 - (embedding <=> $1::vector) AS similarity FROM document_chunks WHERE session_id = $2 AND (1 - (embedding <=> $1::vector)) >= $3 ORDER BY embedding <=> $1::vector ASC LIMIT $4;
```

> **Key Concept: The <=> Operator**
> In pgvector, <=> computes Cosine Distance (range: 0.0 to 2.0). To convert this into Cosine Similarity (where 1.0 is identical and 0.0 is orthogonal), calculate: `1 - (embedding <=> query_vector)`.

Component 2: Embedding Consistency & Rate-Limiting Protection

The Silent Vector Space Mismatch Problem: In vector retrieval, embeddings from different models (even with identical dimensions) occupy completely distinct mathematical spaces. Comparing a `gemini-embedding-001` query against a `gemini-embedding-2` document chunk yields random mathematical noise. The system enforces strict single-model pinning (`gemini-embedding-2`, 768-d) and tracks metadata per row.

src/lib/embeddings.js — Native Batch Embedding & Quota Throttling — JavaScript / Gemini SDK

```
export async function getBatchEmbeddings(texts, options = {}) { const batchSize = Math.min(100, options.batchSize || 50); const ai = getGenAI(); const model = ai.getGenerativeModel({ model: EMBEDDING_MODEL }); const results = []; for (let i = 0; i < texts.length; i += batchSize) { const chunkBatch = texts.slice(i, i + batchSize); // Bundle up to 50 text chunks into a single network call const response = await model.batchEmbedContents({ requests: chunkBatch.map((t) => ({ content: { parts: [{ text: t || " " }] }, outputDimensionality: 768, })), }); // Micro-delay spacing (350ms) keeps throughput safely below 100 RPM quota if (i + batchSize < texts.length) await new Promise((r) => setTimeout(r, 350)); } return results; }
```

Component 3: Sentence-Level Boundary Chunking (No Mid-Sentence Breaks)

Traditional character chunkers slice text at arbitrary offsets (e.g. index 500), chopping sentences in half and severing grammatical subject-verb-predicate relationships. RoastMaster AI leverages native `Intl.Segmenter` to enforce strict linguistic sentence boundaries and guarantees zero page boundary spillover.

src/lib/chunking.js — Grammatically Safe Sentence Chunking — JavaScript / Intl API

```
export function splitTextIntoSentenceChunks(text, options = {}) { const { targetChunkSize = 700, sentenceOverlap = 1, pageNumber = 1 } = options; // Extract atomic sentences respecting abbreviations, quotes, and punctuation const segmenter = new Intl.Segmenter("en", { granularity: "sentence" }); const sentences = Array.from(segmenter.segment(text)).map(s => s.segment.trim()).filter(Boolean); const chunks = []; let currentGroup = [], currentLength = 0; for (const unit of sentences) { if (currentLength + unit.length > targetChunkSize && currentGroup.length > 0) { chunks.push({ pageNumber, content: currentGroup.join(" ") }); // Retain exactly 1 whole sentence as overlap for semantic continuity currentGroup = currentGroup.slice(-sentenceOverlap); currentLength = currentGroup.reduce((acc, s) => acc + s.length + 1, 0); } currentGroup.push(unit); currentLength += unit.length + 1; } if (currentGroup.length > 0) chunks.push({ pageNumber, content: currentGroup.join(" ") }); return chunks; }
```

Component 4: Multi-Signal Composite Reranking & Redundancy Suppression

Pure vector search suffers from semantic drift (e.g., retrieving generic policy chapters instead of specific numbers or part codes). The reranker bridges broad vector recall (Top 15) with high-precision context injection (Top 3–5) using a 4-signal composite scoring function followed by near-duplicate suppression.

src/lib/reranker.js — Multi-Signal Scoring & Diversity Filter — JavaScript

```
// 1. Composite Multi-Signal Formula const compositeScore = Math.min( 1.0, 0.40 * vectorSim +
// 40% Semantic Cosine Vector Distance 0.25 * keywordDensity + // 25% Salient Term Frequency /
Density 0.25 * coverageRatio + // 25% Distinct Query Keywords Matched phraseBonus // 10-18%
Exact Subphrase / Bigram Alignment ); // 2. Hard Relevance Gatekeeper (Discards Noise &
Unrelated Queries) if (chunk.rerankScore < minRelevanceScore) return false; // Default cutoff:
0.36 // 3. Redundancy Suppression (MMR-Style Diversity via Jaccard Overlap) const isDuplicate
= deduplicated.some((existing) => existing.fileName === chunk.fileName &&
calculateTokenOverlap(existing.content, chunk.content) > 0.65 ); if (!isDuplicate)
deduplicated.push(chunk);
```

Component 5: Strict Citation Validation & Anti-Hallucination Engine

Never trust LLM citations at face value. Even state-of-the-art models will hallucinate page numbers, invent nonexistent filenames, or falsely claim a fact is on page 42 when it was never retrieved. RoastMaster AI runs a deterministic post-processing validation layer that verifies model citations against real retrieved chunks before returning to the user.

src/app/api/chat/route.js — Deterministic Post-Generation Citation Verifier — Next.js API Route

```
// Cross-validate candidate citations against actual retrieved chunks for (const cit of
candidateCitations) { const matchingDoc = retrievedDocs.find((doc) => { const pageMatches =
(doc.pageNumber || 1) === parseInt(cit.pageNumber, 10); const fileMatches =
doc.fileName.toLowerCase().includes(cit.fileName.toLowerCase()); return pageMatches &&
fileMatches; }); if (matchingDoc) { // Ensure the cited quote lexically exists in the
retrieved chunk content const quoteWords = extractSalientKeywords(cit.quote); const
matchedWords = quoteWords.filter(w => matchingDoc.content.toLowerCase().includes(w)); if
((matchedWords.length / quoteWords.length) >= 0.35) { verifiedCitations.push({ ...cit,
chunkId: matchingDoc.id }); } } } // If citations failed validation, strip hallucinated
"Source: ..." lines and mark unsupported! if (verifiedCitations.length === 0) {
finalPayload.isDocumentSupported = false; finalPayload.citations = []; finalPayload.suggestion
= finalPayload.suggestion.replace(/Source:s*[^, ]+,s*Pages*d+/gi, "").trim(); }
```

4. RAG Retrieval Evaluation Scorecard & Metrics

Evaluation Results

Automated evaluation executed via `npm run eval:rag` across 30 benchmark queries covering 6 query archetypes:

RECALL@3

96.7%

Target: ≥ 85% (Industry Grade)

MEAN RECIPROCAL RANK (MRR)

0.936

Target: ≥ 0.75 (High Precision)

RECALL@1 (TOP-1 HIT)

90.0%

Target: ≥ 65%

REFUSAL ACCURACY (OOD)

100.0%

Target: 100% (Zero False Citations)

QUERY ARCHETYPE	QUERY COUNT	RECALL@3	MRR	HIT RATE	AVG LATENCY
1. Known Answer (Factual)	6	**100.0%**	0.889	100.0%	730 ms
2. Similar Question (Paraphrased)	5	**100.0%**	0.900	100.0%	608 ms
3. Ambiguous (Underspecified)	5	**100.0%**	1.000	100.0%	596 ms
4. Out-of-Document (Refusal)	5	**100.0%**	1.000	100.0%	586 ms
5. Exact Keyword (Codes/Models)	5	**100.0%**	1.000	100.0%	599 ms
6. Multi-Hop (Cross-Sectional)	4	**75.0%**	0.813	100.0%	595 ms

5. Top 25 Technical Interview Questions & Answers

Interview Questions

Q1 Walk me through the high-level architecture of RoastMaster AI. How do the frontend, CLI, and RAG backend interact?

ARCHITECTURE

RoastMaster AI is architected as a decoupled, multi-client system serving both web and terminal developers:

- **Frontend Clients:** A Next.js 16 (React 19) liquid glass web application and a standalone CLI published on npm (`npx roastmaster-ai`). The CLI extracts local git context (`git diff`, `git status`, `package.json`) and communicates with either Google's Gemini SDK directly or the deployed Next.js API route.
- **Application Server:** Next.js App Router API routes (`/api/chat`, `/api/rag/upload`, `/api/search`) handling validation, session management, and RAG coordination.
- **Storage & Vector Engine:** Aiven PostgreSQL with the `pgvector` extension. It stores user profiles, sessions, conversation history, and 768-dimensional document chunk embeddings within the same relational schema.
- **Inference & Embedding:** Google Gemini 2.5 Flash for standup punchline and architectural generation; `gemini-embedding-2` (768-d) for semantic dense embeddings.

> 💡 **Senior Engineer Soundbite**
> "Rather than building an isolated chatbot, we built a dual-interface developer tool where the CLI meets developers right in their terminal before git push, backed by a unified PostgreSQL pgvector database and Gemini."

Q2 Why did you use Aiven PostgreSQL with pgvector instead of a dedicated vector database like Pinecone or Milvus?

DATABASE

Choosing `pgvector` inside PostgreSQL was an intentional architectural trade-off based on operational simplicity, transactional integrity, and data cohesion:

- **ACID Transactions & Unified State:** In dedicated vector databases (like Pinecone), metadata and text live in one place while users, auth records, and chat history live in another. This creates dual-write problems, eventual consistency headaches, and sync failures. In PostgreSQL, uploading a PDF deletes old chunks and inserts new ones within a single `BEGIN...COMMIT` transaction block.
- **Hybrid Relational Filtering:** With pgvector, vector similarity can be combined in a single SQL statement with relational clauses (e.g., `WHERE session_id = $1 AND user_id = $2`) and PostgreSQL text search (`ILIKE` keyword matching).
- **Cost & Operational Simplicity:** No secondary SaaS bill, no proprietary API lock-in, and zero network hop latency between our relational store and vector index.

> 💡 **Trade-off to Mention**
> "At hundreds of millions of vectors with 10k QPS, dedicated vector databases like Milvus or Qdrant with specialized distributed memory partitioning can outperform pgvector. But for sub-million vector multi-tenant sessions, pgvector gives lower operational overhead, lower latency, and zero dual-write bugs."

Q3 Explain the "Silent Vector Index Corruption" problem you solved in `src/lib/embeddings.js`.

VECTOR SEARCH

This is one of the most dangerous and elusive bugs in vector retrieval systems:

Vector embeddings generated by different models (e.g., `gemini-embedding-001` vs `gemini-embedding-2`, or OpenAI `text-embedding-3-small`) exist in completely separate, non-overlapping coordinate spaces. Even if both models output vectors of identical dimensionality (e.g., 768 floats), a cosine distance comparison between them is mathematically meaningless — yielding pseudo-random similarity scores.

How we solved it:

- **Explicit Schema Tracking:** We altered the database tables to explicitly store `embedding_model VARCHAR(64)` and `embedding_dimension INTEGER` on every row (via `ensureVectorSchema()` in `db.js`).
- **Zero Silent Fallbacks:** We eliminated cross-model fallbacks. If an embedding call fails, it retries with exponential backoff on the *exact same model* rather than silently switching models.
- **Migration Script:** We built `scripts/migrate-embeddings.mjs` to detect any legacy chunks and re-embed them strictly to `gemini-embedding-2`.

Q4 How does your 2-stage retrieval pipeline work? Why not just use top-k from pgvector directly?

RAG RETRIEVAL

Dense vector search (Cosine Distance in pgvector) is fantastic at **broad semantic recall** (understanding that "damaged screen" relates to "accidental protection"), but it suffers from **semantic drift** and lacks precision for exact keywords, model numbers, codes, or specific clause matching.

If you feed 15 raw vector chunks directly to Gemini, you encounter **Lost in the Middle** attention degradation, higher token costs, and increased hallucination risk.

Our 2-Stage Architecture:

1. **Stage 1 (Broad Recall):** Query pgvector with cosine distance to retrieve 15 candidate chunks (low threshold: 0.20).
2. **Stage 2 (High-Precision Multi-Signal Reranking):** We score candidates across 4 dimensions:
   - 40% Vector Cosine Similarity
   - 25% Salient Keyword Density (stopwords excluded)
   - 25% Query Keyword Coverage Ratio
   - 10% Exact Subphrase / Bigram Alignment Bonus
3. **Relevance Gating & Diversity:** Any chunk below 0.36 composite score is dropped. Chunks with > 0.65 Jaccard token overlap are suppressed to eliminate redundancy, delivering the Top 3–4 pristine chunks to Gemini.

Q5 Walk me through your chunking strategy in `src/lib/chunking.js`. Why sentence-level over fixed-character chunking?

RAG CHUNKING

Fixed-character chunking (e.g. splitting every 500 characters) cuts text at arbitrary byte offsets. This frequently splits words, numbers, or grammatical clauses in half (e.g., cutting "The warranty limit is $500" into "...warranty limit is $" and "500..."). This destroys the semantic embedding vector and guarantees retrieval failure.

Our Implementation (`splitTextIntoSentenceChunks`):

- Uses native JavaScript `Intl.Segmenter("en", { granularity: "sentence" })`, which natively understands English grammar, abbreviations (e.g., "Mr.", "Inc.", "i.e."), quotes, and terminal punctuation.
- Assembles complete sentences into a target budget of ~700 characters (~120–160 words).
- Maintains a `sentenceOverlap = 1` (exactly 1 complete sentence carried over to the next chunk) to preserve context continuity across chunk boundaries.
- **Zero Page Boundary Spillover:** When processing PDFs, `splitPagesIntoChunks` guarantees that chunks never merge across page boundaries, preserving exact 1-to-1 page attribution for citations.

Q6 How does RoastMaster prevent citation hallucinations? Can you trust an LLM when it returns citations?

ANTI-HALLUCINATION

Never trust LLM citations blindly. LLMs are probabilistic token predictors, not database indexers. When an LLM is asked to cite a page number, it frequently hallucinates believable numbers (e.g., "Source: manual.pdf, Page 42") even when page 42 was never in the context window.

Our Deterministic Citation Validation Layer (`src/app/api/chat/route.js`):

1. **Negative Refusal Check:** If the model's text contains refusal phrases like "couldn't find this in your document" or "not mentioned", we immediately force `isDocumentSupported = false` and clear all citations.
2. **Grounding Cross-Verification:** For every candidate citation in the JSON, our code scans the *actual retrieved chunks*. It checks:
   - Does the cited `pageNumber` match a retrieved chunk?
   - Does the cited `fileName` match?
   - Does the cited `quote` have at least 35% salient keyword overlap with the actual chunk text?
3. **Hallucination Stripping:** If a citation fails this test, it is discarded. If no citations remain valid, any hallucinated "Source: ..." lines are stripped from the response text using regex, preventing fake sources from reaching the user.

Q7 Explain the Conversational Banter Detector (`isConversationalBanter`). Why is it critical for RAG user experience?

UX & SEARCH

In standard RAG systems, every single message sent by a user triggers a vector search against the document index. This creates two major problems:

- **Unnecessary Latency & Token Waste:** If a user says "lol that was funny", "bro", "huh?", or "roast me", vector searching the user's PDF policy document is pointless and wastes 300–600ms.
- **Inappropriate Bot Apologies:** A naive RAG bot will respond to "haha nice" with: "*I apologize, but I could not find information regarding 'haha nice' in your document policy.pdf.*" This completely ruins conversational flow and comedy crowd work.

Our Solution: `isConversationalBanter()` analyzes the query against a calibrated set of interjections, heckles, and short conversational commands. If detected, RAG retrieval is bypassed completely, letting Gemini respond purely in its sharp standup persona without searching or apologizing for missing document context.

Q8 How does the system stay below Google Gemini's 100 RPM Free Tier quota during bulk document ingestion?

PERFORMANCE

A typical 20-page PDF document produces 80 to 120 chunks. If you send each chunk as a separate HTTP request to Gemini's `embedContent` endpoint, you immediately hit the `429 RESOURCE_EXHAUSTED` rate limit (100 RPM ceiling).

Our 3-Tier Mitigation Strategy:

- **Native Batching (`batchEmbedContents`):** We bundle up to 50 text chunks into a single API payload. A 100-chunk document takes only 2 network calls instead of 100!
- **Cadence Micro-Delays:** We insert a 350ms throttle between batches to smooth out request spikes.
- **Exponential Backoff with Header Parsing:** Our `extractRetryDelayMs` function parses Google's RPC error string (e.g. "*Please retry in 2.6s*") and sleeps for the exact recommended duration before retrying.
- **In-Memory LRU Cache:** Repeated query embeddings are stored in a 500-item memory cache, saving API quota entirely on popular questions.

Q9 Explain the difference between HNSW and IVFFlat index types in pgvector. What did you configure and why?

VECTOR INDEXING

`pgvector` supports two primary Approximate Nearest Neighbor (ANN) index types:

- **IVFFlat (Inverted File Flat):** Divides vector space into Voronoi cells (clusters) using k-means. Queries search only vectors in the closest cells (`probes`).
  Pros: Fast build time, small memory footprint.
  Cons: Requires training data; recall degrades if vectors are added after training; lower recall-vs-speed curve.
- **HNSW (Hierarchical Navigable Small World):** Builds a multi-layer graph where bottom layers contain all vectors and top layers contain long-range skips.
  Pros: Outstanding recall (> 98%), faster queries than IVFFlat, handles dynamic incremental inserts without index retraining.
  Cons: Higher RAM usage and slower initial index build.

RoastMaster's Choice: We configured **HNSW** with cosine distance (`vector_cosine_ops`). Because chat sessions and documents are added incrementally by users, HNSW's ability to maintain high recall without periodic rebuilds made it the superior choice.

Q10 How does hybrid session search work in `src/app/api/search/route.js` and `searchSimilarSessions`?

HYBRID SEARCH

When searching historical chat sessions, pure keyword search fails on concepts (e.g. searching "API errors" misses a session titled "Stripe Nil Exception"), while pure vector search fails on exact tokens (e.g. session IDs, specific package names).

Our Hybrid SQL Scoring Function:

```
ROUND(GREATEST( CASE WHEN embedding IS NOT NULL THEN (1 - (embedding <=> $1::vector)) ELSE 0
END, CASE WHEN title ILIKE '%' || $2 || '%' THEN 0.88 ELSE 0 END, CASE WHEN
REPLACE(REPLACE(title, '-', ''), ' ', '') ILIKE '%' || $6 || '%' THEN 0.86 ELSE 0 END, CASE
WHEN snippet ILIKE '%' || $2 || '%' THEN 0.78 ELSE 0 END, CASE WHEN category ILIKE '%' || $2
|| '%' THEN 0.68 ELSE 0 END )::numeric, 4) AS "similarityScore"
```

PostgreSQL calculates the greatest score among vector similarity, title match, normalized whitespace/hyphen-stripped match, and snippet match, classifying the result as `'hybrid'`, `'keyword'`, or `'semantic'`.

Q11 Explain the evaluation metrics you measured in `rag_evaluator.js`. What is MRR and why does it matter?

EVALUATION

We measured 5 core industry metrics across 30 benchmark queries:

- **Recall@K:** The fraction of queries where at least one ground-truth chunk was successfully retrieved within the top K results. RoastMaster achieved **Recall@3 = 96.7%** and **Recall@5 = 100%**.
- **MRR (Mean Reciprocal Rank):** Measures *where* the first relevant document appears in the ranked list. If the correct chunk is ranked #1, reciprocal rank is $1/1 = 1.0$. If it's ranked #2, it's $1/2 = 0.50$. Formula: $ ext{MRR} = rac{1}{|Q|} sum_{i=1}^{|Q|} rac{1}{ ext{rank}_i}$. RoastMaster achieved an outstanding **MRR of 0.936** (target: $ge 0.75$), indicating that the correct chunk is almost always at Rank #1.
- **Precision@3 (In-Doc):** Proportion of the top 3 retrieved chunks that are actually relevant (achieved **74.7%**), proving low context noise.
- **Refusal Accuracy (Negative Grounding):** For out-of-document queries (where the answer does not exist), the retriever must return 0 chunks. RoastMaster scored **100%**.

Q12 What is "Negative Grounding" or "Safe Refusal", and why is it essential for enterprise RAG?

ANTI-HALLUCINATION

Most RAG tutorials only evaluate "happy path" queries where the answer exists in the document. In real life, users frequently ask questions that are *completely absent* from their uploaded document (e.g., asking for the school cafeteria lunch menu from a laptop policy manual).

Without negative grounding, the vector search will force the "least bad" 3 chunks into the LLM context. The LLM will then either hallucinate an answer or fabricate a citation to an unrelated policy.

How RoastMaster handles Out-of-Document (OOD) queries:

- Our reranker's hard relevance threshold (0.36) filters out low-scoring irrelevant chunks, returning an empty candidate list (0 chunks).
- When 0 chunks are returned, our chat prompt enforces a strict refusal protocol: explicitly stating *"I couldn't find this in your document"*, returning `isDocumentSupported: false`, and outputting an empty citations array `[]`.

Q13 How does your Hybrid Resilience Fallback work when the vector search API is down or throttled?

RESILIENCE

If Google's embedding API experiences a 503 outage or rate-limiting spike, query embedding generation will fail. In a naive RAG system, the entire search pipeline crashes with an unhandled exception.

Our Resilient Fallback (`src/lib/reranker.js:retrieveAndRerankChunks`):

- If `initialCandidates` is empty (vector search failed or returned 0 hits), the system triggers `getSessionChunksDirect(sessionId, 50)` to pull direct raw document chunks from PostgreSQL.
- These chunks are assigned a neutral baseline similarity (0.50).
- The multi-signal reranker runs with an adjusted cutoff threshold (0.24). The salient keyword density, query coverage ratio, and exact phrase matching signals kick in to score and rank chunks lexically.
- This ensures that even during a complete third-party embedding API outage, users can still ask questions containing keywords, codes, or phrases and receive grounded answers!

Q14 How do you optimize latency so the user doesn't wait for background database operations?

LATENCY OPTIMIZATION

In conversational AI, Time-to-First-Token (TTFT) and Total Turnaround Time dictate user perception of speed. We implemented several optimizations:

- **Asynchronous Fire-and-Forget Embedding:** At the end of every chat turn, we embed the conversation exchange and update `session_embeddings`. Instead of `await`ing this operation before returning the HTTP response, we trigger it as a background promise (`getEmbedding(...).then(...).catch(...)`) and immediately return the JSON response to the user.
- **Character-by-Character Teleprompter (`useCharacterReveal`):** Instead of an abrupt text dump or waiting for a sluggish server-sent events stream over poor mobile networks, the client renders the structured response smoothly at 2 characters every 18ms, mimicking a live standup teleprompter.
- **Broad Recall Candidate Capping:** Broad vector search fetches exactly 15 candidates — large enough for 100% recall, but small enough to rerank in under 5 milliseconds in Node.js memory.

Q15 Explain how the CLI tool (`bin/cli.mjs`) inspects git repositories and formats terminal output.

CLI ARCHITECTURE

The CLI is designed to be zero-dependency and executable anywhere via `npx roastmaster-ai`:

- **Git Context Extraction:** Using `child_process.execSync`, it checks:
  - `git rev-parse --is-inside-work-tree` to verify git context.
  - `git status --short` to see untracked/modified files.
  - `git log -1 --pretty=format:"%s (%h)"` for the last commit message.
  - `git diff HEAD` and `git diff --staged` (capped at 3,500 characters) to analyze uncommitted code changes.
  - Inspects `package.json` for suspicious dependencies, version counts, and build scripts.
- **Local & Remote Fallback:** If a local `GEMINI_API_KEY` is present, it computes the roast locally. If not, it falls back to the live deployed Vercel API endpoint.
- **ANSI Terminal Box Formatter:** Hand-crafted ANSI escape sequences render rounded borders, hot pink headers, severity flame meters, and Backstage Real Talk callouts formatted to terminal width.

Q16 How did you design the LLM System Prompt to guarantee comedy freshness and prevent stock clichés?

PROMPT ENGINEERING

Standard LLM prompts default to predictable internet tropes (e.g. "That's like an umbrella in a car wash", "Did your cat walk on the keyboard?").

Our Prompt Defense Mechanisms (`src/app/api/chat/route.js`):

- **Length Constraints:** Strictly limited to 1–2 punchline sentences (under 35–45 words). No wandering monologues.
- **Negative Prompting / Banned Clichés:** Explicitly bans stale metaphors ("umbrella in a car wash", "dog chasing tail") and formulaic openings ("Bhai tu seriously...", "Matlab tu wahi banda hai...").
- **Audience Profile Injection:** When a user sets a VIP profile (stage title, favorite tech, requested roast level: mild/sarcastic/incineration), their quirks and tech preferences are dynamically injected into the system instruction, enabling customized comedy crowd work callbacks.
- **Structured Output Guarantee:** A strict JSON schema is enforced (with `title`, `roast`, `severity`, `category`, `suggestion`, `citations`), backed by regex fallback parsing in case of markdown fencing.

Q17 What was the migration from Firestore to PostgreSQL in `scripts/migrate-firestore-to-postgres.mjs`? Why migrate?

ARCHITECTURE MIGRATION

The project initially started with Google Cloud Firestore (NoSQL) for user profiles and chat sessions. As the RAG requirements matured, Firestore presented critical limitations:

- **No Native Vector Search:** Firestore lacked robust, integrated HNSW vector indexing for pgvector-style similarity operations.
- **Complex Queries & Aggregations:** Performing combined keyword searches, group-by document counts, and relational joins across user profiles and document chunks in Firestore required multiple client roundtrips and denormalized data.
- **Cost Model:** Firestore charges per read/write document. When a RAG search evaluates candidate documents and sessions, read operations scale linearly with cost. PostgreSQL provides predictable database compute pricing with superior query capabilities.

We wrote `migrate-firestore-to-postgres.mjs` to authenticate via Firebase SDK, extract collections and subcollections across sessions and messages, and stream them into PostgreSQL with automatic foreign key resolution.

Q18 Explain Redundancy Suppression in your reranker. How is it implemented and why is it needed?

RAG RERANKING

When chunking documents with sentence overlap (e.g. 1-sentence overlap), consecutive chunks often contain nearly identical wording. If a query matches both chunks, naive top-k retrieval selects Chunk #3 and Chunk #4, wasting two of the four context slots on duplicate information.

Our Solution: Jaccard Lexical Overlap Suppression:

- For each candidate passing the relevance gatekeeper, we calculate token overlap against already accepted chunks: $$ ext{Jaccard Overlap} = rac{|A cap B|}{|A cup B|}$$
- If overlap exceeds **0.65 (65%)** within the same file, the chunk is identified as a near-duplicate and skipped.
- This mimics Maximal Marginal Relevance (MMR) diversity, ensuring that every retrieved chunk delivers net-new facts to the LLM.

Q19 How would you scale this architecture to handle 500,000 active daily users and millions of documents?

SYSTEM DESIGN

To scale from prototype/freelance scale to enterprise production:

1. **Asynchronous Document Ingestion Queue:** Replace direct synchronous HTTP PDF upload with an asynchronous message queue (e.g., AWS SQS or BullMQ / Redis). Users receive a job ID immediately; background worker pools handle OCR, chunking, and embedding generation.
2. **PostgreSQL Read Replicas & Connection Pooling:** Implement PgBouncer for transaction-level pooling and route vector search queries to read replicas.
3. **Dedicated Vector Partitions:** Partition the `document_chunks` table by `user_id` or `session_id` (PostgreSQL declarative partitioning) so HNSW indexes remain resident in RAM.
4. **Edge Caching:** Cache repeated query embeddings in Redis with TTLs instead of single-instance memory.

Q20 What are the 6 query archetypes you evaluated, and why is this categorization important?

BENCHMARKING

Evaluating RAG systems only on simple factual questions produces misleadingly optimistic benchmarks. We defined and benchmarked 6 archetypes:

1. **Known Answer (Factual):** Explicit questions directly answered in text ("What percentage of part cost is covered?").
2. **Similar Paraphrase:** Semantically identical queries with zero overlapping vocabulary ("If I knock over coffee on my keyboard...").
3. **Ambiguous (Underspecified):** Short 2-3 word queries lacking syntax ("laptop damage fee").
4. **Out-of-Document (Refusal):** Queries completely outside document domain ("cafeteria lunch menu"). Tested for safe refusal.
5. **Exact Keyword:** Technical model codes and emails ("Lenovo Yoga 12 Thinkpad", "helpdesk@kingsacademy.edu.jo").
6. **Multi-Hop:** Requires synthesizing facts across disparate sections/pages ("If my laptop requires extensive repair...").

Q21–25 Rapid-Fire Technical Production Questions

QUICK-FIRE

Q21: How do you handle PDF parsing without losing page numbers?
We use `unpdf` with `extractText(uint8Array, { mergePages: false })`. This returns an array of strings where index `i` maps strictly to Page `i + 1`, preventing text from different pages from bleeding together.

Q22: Why did you implement model fallback cascading in `api/chat/route.js`?
If a specific Gemini model (e.g. `gemini-flash-lite-latest`) encounters temporary 503 capacity issues, the route automatically cascades to `gemini-2.5-flash-lite` and `gemini-3.5-flash-lite` before failing, guaranteeing 99.9% API uptime.

Q23: What security protections exist for database connections in `db.js`?
The connection string parses out insecure flags, uses SSL with `rejectUnauthorized: false` for Aiven cloud certificates, sets a maximum pool size of 10 connections, and uses parameterized queries (`$1, $2, ...`) across 100% of SQL statements to eliminate SQL injection vulnerabilities.

Q24: What is the VisionOS Liquid Glass UI design system in the frontend?
A hardware-accelerated CSS design system combining Tailwind CSS v4, multi-layered backdrop filters (`backdrop-blur-md`), translucent dark borders (`rgba(255, 255, 255, 0.08)`), and dynamic SVG metaball symbiote goo animations that run at 60 FPS.

Q25: What is the single biggest lesson you learned building RoastMaster AI?
"In generative AI, 80% of your bugs are retrieval and data integrity problems masquerading as prompt problems. Standardizing on an explicit embedding model, enforcing sentence-level chunking, and implementing a deterministic citation validation layer did 10x more for answer quality than tweaking LLM temperature."

6. Cheat Sheet & High-Impact Interview Soundbites

Quick Recall

The 60-Second "Elevator Pitch"

> "RoastMaster AI is a full-stack developer tool available as both a Next.js 16 web application and an npm CLI package. It acts as an unfiltered standup comedian and senior-staff code reviewer. Users can upload technical documentation, and our system retrieves exact passages using an end-to-end RAG pipeline built on Aiven PostgreSQL and pgvector with HNSW indexing.
>
> We built a two-stage retrieval engine: broad vector recall retrieves the top 15 candidates, followed by a multi-signal composite reranker that evaluates keyword density, query coverage, and phrase alignment down to the top 3 chunks. In benchmark evaluations across 30 queries and 6 archetypes, our pipeline achieved a 96.7% Recall@3, an MRR of 0.936, and 100% Safe Refusal on out-of-document queries."

Key Numbers & Constants to Memorize

METRIC / CONSTANT	EXACT VALUE	ARCHITECTURAL RATIONALE
Embedding Dimension	**768 dimensions**	Native dimensionality of `gemini-embedding-2`.
Target Chunk Size	**~700 characters**	~120–160 words; fits 2–4 full grammatical sentences per chunk.
Sentence Overlap	**1 complete sentence**	Preserves cross-chunk context without token bloat.
Batch Embeddings Size	**50 chunks / request**	Stays safely below Gemini's 100 RPM Free Tier limit.
Reranker Weights	**40% / 25% / 25% / 10%**	Vector Cosine / Keyword Density / Query Coverage / Phrase Bonus.
Relevance Cutoff	**0.36 threshold**	Discards unrelated noise; triggers 100% safe refusal on OOD queries.
Redundancy Cutoff	**0.65 Jaccard overlap**	Suppresses near-duplicate consecutive chunks.
Evaluated Recall@3	**96.7%**	Exceeds industry standard benchmark of ≥ 85%.
Evaluated MRR	**0.936**	Average rank of first correct chunk is #1.07.

The "Toughest Challenge" Story (STAR Method)

Situation: Early in testing, our RAG system experienced subtle citation hallucinations where the model cited real filenames but nonexistent page numbers, and occasionally returned random chunks for technical part queries.
Task: Eliminate citation hallucinations and diagnose the retrieval inaccuracy without adding excessive latency.
Action: I discovered two root causes: First, cross-model embedding contamination—different files were embedded with different model versions, creating silent index corruption. I standardized the pipeline strictly to `gemini-embedding-2` with explicit database column schema enforcement. Second, I introduced a deterministic post-generation citation validation layer in the API route that cross-checks the

LLM's cited page and quote against actual retrieved chunks before serving the user. If ungrounded, fake sources are stripped automatically.
Result: Citation accuracy hit 100%, out-of-document refusal accuracy reached 100%, and MRR improved from 0.72 to 0.936.
